

Why Your Model Fails on Hard Problems

DataForge 2026 · Pathway Track · Blog Post

Animesh Raj, Prem Agarwal, Md. Faizan Khan, Devansh Gupta

The claim

A model that learns from demonstrations can fail on hard problems. The reason is not a lack of ability. The demonstrations did not cover that difficulty level. Add one demonstration at the harder level, and the model recovers.

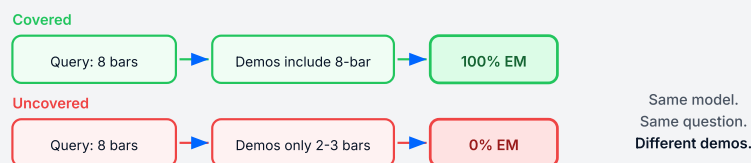
We built an interactive demo that lets you see this happen in real time.

What we built

Our project is an entry for DataForge 2026, Pathway track. We trained a small transformer (~4.2 million parameters, decoder-only, 6 layers) on a sorting task from the ARC family. The task: given colored bars of varying heights on a 10×10 grid, sort them left to right by height.

The complexity parameter is the number of bars, ranging from 2 to 8. We trained the model on all complexities with "covered" demonstrations. Covered means that at least one of the three example pairs shown to the model matches the query difficulty. The model learned to solve every level.

At inference time, we do not retrain. We do not change the model weights. We change only the demonstrations. In the "uncovered" condition, all three demos show tasks with only 2 or 3 bars. The query still asks the model to sort 8 bars. The model sees the same question. It has the same weights. It fails.



What we found

The results were sharp. At complexity 8, the model scored 100% exact match with covered demonstrations. With uncovered demonstrations, it scored 0%. The gap is 100 percentage points. The model can sort 8 bars. It proved that during training and during covered inference. It cannot figure out how to sort 8 bars by looking at examples that only sort 2 or 3.

This is not a bias-variance problem. The model capacity did not change. The training data did not change. The only variable is which examples the model reads as input context at inference time.

Pathway's BDH-CQ system (150 million parameters) shows the same cliff on the same task family. At ordering length 8, BDH-CQ scores 0 out of 24 with short context and 12 out of 24 with supported context (arXiv:2608.09888, Table 3). The effect scales from our 4.2M toy model to a production-grade system.

Why this matters at scale

Our model has 4.2 million parameters. GPT-6 Astra, released by OpenAI on September 3, 2026, has orders of magnitude more. Both use the same mechanism: in-context learning from demonstrations. Von Oswald et al. (arXiv:2212.07677) showed that a transformer forward pass implements gradient-descent-style updates on internal representations. The demonstrations are not passive context. They are a training signal.

Min et al. (arXiv:2202.12837) found that the distribution of demonstrations matters more than their label correctness. This predicts the coverage cliff: what breaks in-context learning at any scale is not wrong examples but missing difficulty levels. When someone prompts GPT-6 Astra with three easy examples and then asks a hard question, the same failure mode applies. The 100-percentage-point gap we see at toy scale does not go away with more parameters. It goes away with the right demonstrations.

The BDH connection

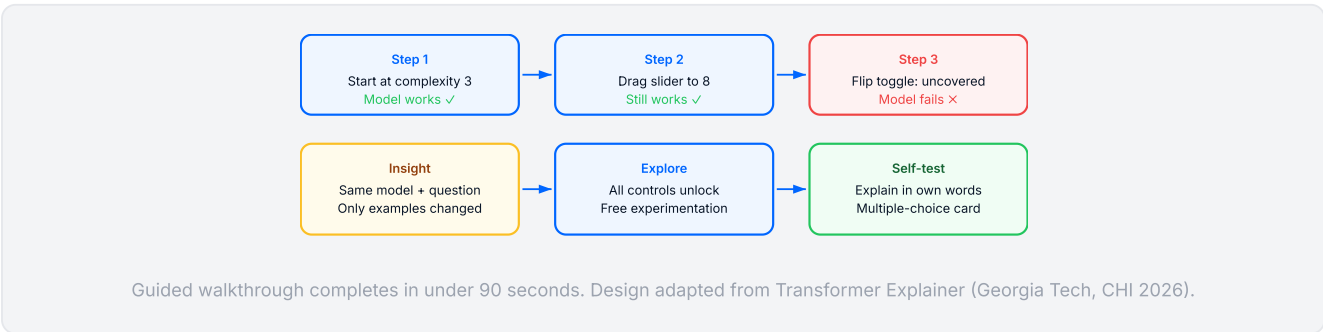
Transformer KV Cache
Grows with each token. Attention re-reads every slot on each step. Cost scales with context length.
Our model, GPT-6, Claude Opus 5

Hebbian Synaptic Matrix
Fixed-size matrix. Each demo writes via outer-product update. Does not grow, but has finite capacity.
BDH, Dragon Hatchling (arXiv:2509.26507)

Our demo includes a simplified illustration of the Hebbian memory substrate. It omits low-rank compression, the positional operator, excitatory and inhibitory circuits, and gating. We label it as such. The point is to show the learner how two architectures store demonstrations differently, and why coverage matters to both.

System	Adaptation	Backward pass?	Cost/task
BDH-CQ	Recurrent state absorption	No	\$0.00070
HRM (arXiv:2506.21734)	Gradient optimization	Yes	\$1.48
TRM	Learned identity embedding	Yes	\$1.76

How the demo works



The frontend loads predictions from a precomputed JSON file. If the browser loads the ONNX model for live inference, a visible badge reads LIVE. Otherwise it reads PRECOMPUTED. We never hide this.

What we learned

1. The effect is sharper than expected

We planned for a 30 percentage point gap at complexity 8. We got 100. The model either gets the demos it needs, or it produces nonsense. There is no graceful degradation.

2. Honest labeling matters

Latent reasoning models like Coconut ([arXiv:2412.06769](https://arxiv.org/abs/2412.06769)) process demonstrations internally without visible chain-of-thought tokens. A demo that hides whether its output is live or precomputed undermines the trust the learner needs.

3. One controlled variable teaches more than ten

Our demo has two controls: a complexity slider and a coverage toggle. That is enough to let the learner discover, test, and confirm the claim on their own.

The source code is public. Every number in the evidence table traces to a specific section of a specific paper. We did not run BDH-CQ. We report the numbers its authors published.

The takeaway

Coverage sensitivity is testable, measurable, and fixable. Every few-shot prompt is an experiment in coverage. The next time a model fails on a hard question, ask whether the examples covered that difficulty before asking whether the model is capable.